Prototype2Code

De la maquette UI à l'Arbre de Mise en Page (UI Layout Tree)

Explication de la méthode décrite en section 3.2 et formalisée par l'Algorithme 1 : construction de la hiérarchie, puis segmentation Row/Column pour le modèle Flexbox.

L'objectif du processus est de transformer une maquette (des rectangles positionnés en pixels, sans structure logique) en un **Arbre de Mise en Page (UI Layout Tree)** organisé en groupes imbriqués, façon « poupées russes ». Cette structure porte ensuite directement les instructions nécessaires à la génération de code HTML (imbrication de `<div>`) et CSS (Flexbox ou positionnement absolu). Le processus se déroule en deux grandes phases.

Phase 1 — Initialisation de l'arbre et gestion de la hiérarchie

Avant de construire l'arbre, le système exploite les résultats de l'étape de « linting » (correction) pour préparer des données propres.

1. Reconstruction des relations de contenance

- Les groupes perceptuels détectés (cartes, barres de navigation, etc.) sont triés par taille **ascendante**, du plus petit au plus grand.
- Pour chaque couche, on calcule l'IoU (Intersection sur Union) afin de vérifier si elle appartient géométriquement à un groupe ; si oui, elle lui est rattachée.
- Les éléments fragmentés (ex. les morceaux d'une icône coupée par la détection) sont fusionnés en une seule entité avec de nouvelles coordonnées globales, du point haut-gauche au point bas-droit.

2. Construction de l'arbre initial (Algorithme 1, Phase 1)

- Extraction de toutes les couches « feuilles » — les éléments terminaux comme le texte ou les images.
- Ces éléments sont triés par taille **descendante**, du plus grand au plus petit.
- Chaque élément est inséré sous le nœud qui le contient physiquement et qui possède la plus petite taille possible — garantissant une hiérarchie précise, du type « poupée russe ».

3. Gestion des chevauchements

- Pour chaque nœud, le système vérifie si ses enfants se superposent.
- En cas de chevauchement, un nouveau nœud parent est créé avec l'attribut `need_absolute=True`, indiquant que ces éléments devront utiliser un positionnement CSS absolu plutôt que le flux normal.

Phase 2 — Segmentation Row/Column pour le modèle Flexbox

Une fois la hiérarchie de base établie, il faut organiser les éléments de façon ordonnée pour permettre un design adaptatif (responsive).

1. Segmentation verticale (lignes)

- L'algorithme cherche les coordonnées Y où une ligne horizontale peut traverser toute la zone sans couper aucun élément.
- Cela divise l'espace en segments verticaux — des « étages ».

2. Segmentation horizontale (colonnes)

- À l'intérieur de chaque segment vertical, le système cherche des coordonnées X pour tracer des lignes verticales sans couper d'éléments.
- Cela permet de regrouper les éléments côte à côte sur une même ligne.

3. Récursion et attribution Flexbox

- Les deux étapes précédentes sont appliquées récursivement jusqu'à ce qu'il soit impossible de segmenter davantage.
- Si plusieurs éléments sont dans le même segment horizontal : le parent reçoit `flex-direction: row`.
- S'ils sont alignés verticalement : le parent reçoit `flex-direction: column`.

Synthèse des deux phases

Phase	Objectif	Sortie produite
1. Initialisation de la hiérarchie	Déterminer les relations de contenance et fusionner les éléments fragmentés	Arbre brut « poupée russe » + nœuds `need_absolute=True`
2. Segmentation Flexbox	Découper récursivement l'espace en lignes puis colonnes	Arbre final avec `flex-direction: row / column` sur chaque parent

Résultat final

À l'issue de ce processus, on obtient un arbre structuré où chaque nœud porte des instructions explicites pour la génération de code : imbrication HTML (des `<div>` logiquement groupés) et modèle de boîte CSS (Flexbox en `row`/`column`, ou positionnement absolu lorsque nécessaire). Cette organisation en « groupes imbriqués » rend le code final bien plus maintenable et lisible qu'un placement au pixel près.